

MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY

JAIPUR

Department of Computer Science and Engineering

PROJECT REPORT

on

**“Scale-MIA: A Scalable Model Inversion Attack against
Secure Federated Learning via Latent Space Reconstruction”**

Submitted by

Samay Chourasia (2023UCP1851)

Nikhil Kumar Pippal (2023UCP1979)

VI Semester, B.Tech. (Computer Science and Engineering)

Under the guidance of

Dr. Ramesh Babu Battula

Academic Year 2025–2026

Abstract

This project reproduces the research paper “Scale-MIA: A Scalable Model Inversion Attack against Secure Federated Learning via Latent Space Reconstruction.” The paper demonstrates that private client images can be reconstructed from federated learning updates even under secure aggregation.

The implementation was reproduced using the official PyTorch artifact under WSL with GPU acceleration. Experiments were conducted on the FashionMNIST dataset using multiple reconstruction batch sizes ranging from 64 to 1024.

The reproduced results validate the scalability and efficiency claims of Scale-MIA. Additional engineering improvements were implemented, including scalable visualization support, dynamic tensor saving, organized experiment logging, and reconstruction comparison generation.

1. Introduction

Federated Learning (FL) is a distributed machine learning paradigm where clients collaboratively train a model without sharing raw data. Although FL is generally considered privacy-preserving, recent studies show that gradients and aggregated updates may still leak sensitive information.

Scale-MIA is a scalable model inversion attack capable of reconstructing private client training samples from federated learning updates. Unlike traditional gradient inversion attacks, Scale-MIA performs latent-space reconstruction, making the attack significantly more scalable and computationally efficient.

2. Objectives

The main objectives of this project are:

1. Reproduce the Scale-MIA attack pipeline.
2. Reproduce FashionMNIST experiments from the original paper.
3. Validate scalability claims using multiple reconstruction batch sizes.
4. Generate quantitative and qualitative reconstruction results.
5. Analyze reproducibility deviations and implementation challenges.

3. Methodology

The Scale-MIA attack pipeline works as follows:

1. A malicious federated learning server distributes a modified global model.
2. Clients train locally using private data.
3. Clients send aggregated updates back to the server.
4. The malicious server analytically reconstructs latent representations.
5. A decoder reconstructs the original client images from the latent space.

The implementation consists of:

- Encoder Network
 - Decoder Network
 - Federated Averaging Setup
 - Latent Space Reconstruction
 - Reconstruction Evaluation Metrics
-

4. Experimental Setup

Hardware

- NVIDIA RTX 4060 Laptop GPU
- WSL Ubuntu Environment
- INTEL Core i7 12th Gen
- 16 GB RAM

Software

- Python
- PyTorch
- Torchvision
- CUDA
- VS Code

Dataset

- FashionMNIST
- CIFAR
- HMNIST

Experimental Parameters

- Reconstruction batch sizes: 64, 128, 256, 512, 1024
- Iteration setting: FedSGD (iter=1)
- Test rounds: 3

```

saman@Sam-AsusTuf-F15: ~/ | × + v
saman@Sam-AsusTuf-F15:~$ mkdir -p ~/projects
saman@Sam-AsusTuf-F15:~$ cd ~/projects
saman@Sam-AsusTuf-F15:~/projects$ ls
saman@Sam-AsusTuf-F15:~/projects$ git clone https://github.com/unknown123489/Scale-MIA.git
Cloning into 'Scale-MIA'...
remote: Enumerating objects: 389, done.
remote: Counting objects: 100% (389/389), done.
remote: Compressing objects: 100% (384/384), done.
remote: Total 389 (delta 149), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (389/389), 73.56 MiB | 292.00 KiB/s, done.
Resolving deltas: 100% (149/149), done.
saman@Sam-AsusTuf-F15:~/projects$ cd Scale-MIA
saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ python3 -m venv venv
saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ source venv/bin/activate
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.10/site-packages (22.0.2)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 327.3 kB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 22.0.2
    Uninstalling pip-22.0.2:
      Successfully uninstalled pip-22.0.2
  Successfully installed pip-26.1.1

```

```

(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
Looking in indexes: https://download.pytorch.org/whl/cu121
Collecting torch
  Downloading torch-2.5.1%2Bcu121-cp310-cp310-linux_x86_64.whl (780.4 MB)
    780.4/780.4 MB 6.0 MB/s 0:07:39
Collecting torchvision
  Downloading torchvision-0.20.1%2Bcu121-cp310-cp310-linux_x86_64.whl (7.3 MB)
    7.3/7.3 MB 10.5 MB/s 0:00:00
Collecting torchaudio
  Downloading torchaudio-2.5.1%2Bcu121-cp310-cp310-linux_x86_64.whl (3.4 MB)
    3.4/3.4 MB 8.8 MB/s 0:00:00
Collecting filelock (from torch)
  Downloading filelock-3.25.2-py3-none-any.whl.metadata (2.0 kB)
Collecting typing_extensions>=4.8.0 (from torch)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting networkx (from torch)
  Downloading networkx-3.4.2-py3-none-any.whl.metadata (6.3 kB)
Collecting jinja2 (from torch)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting fsspec (from torch)
  Downloading fsspec-2026.2.0-py3-none-any.whl.metadata (10 kB)
Collecting nvidia-cuda-nvrtc-cu12==12.1.105 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (23.7 MB)
    23.7/23.7 MB 2.3 MB/s 0:00:10
Collecting nvidia-cuda-runtime-cu12==12.1.105 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (823 kB)
    823.6/823.6 kB 1.3 MB/s 0:00:00
Collecting nvidia-cuda-cupti-cu12==12.1.105 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (14.1 MB)
    14.1/14.1 MB 1.4 MB/s 0:00:09
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
    664.8/664.8 MB 5.2 MB/s 0:02:33
Collecting nvidia-cublas-cu12==12.1.3.1 (from torch)
  Downloading nvidia_cublas_cu12-12.1.3.1-py3-none-manylinux1_x86_64.whl (410.6 MB)
    410.6/410.6 MB 6.7 MB/s 0:01:09

```

```

Collecting nvidia-cuda-nvrtc-cu12==12.1.105 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (23.7 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.7/23.7 MB 2.3 MB/s 0:00:10
Collecting nvidia-cuda-runtime-cu12==12.1.105 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (823 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 823.6/823.6 kB 1.3 MB/s 0:00:00
Collecting nvidia-cuda-cupti-cu12==12.1.105 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (14.1 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.1/14.1 MB 1.4 MB/s 0:00:09
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 5.2 MB/s 0:02:33
Collecting nvidia-cublas-cu12==12.1.3.1 (from torch)
  Downloading nvidia_cublas_cu12-12.1.3.1-py3-none-manylinux1_x86_64.whl (410.6 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 410.6/410.6 MB 6.7 MB/s 0:01:09
Collecting nvidia-cufft-cu12==11.0.2.54 (from torch)
  Downloading nvidia_cufft_cu12-11.0.2.54-py3-none-manylinux1_x86_64.whl (121.6 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 121.6/121.6 MB 5.1 MB/s 0:00:23
Collecting nvidia-curand-cu12==10.3.2.106 (from torch)
  Downloading nvidia_curand_cu12-10.3.2.106-py3-none-manylinux1_x86_64.whl (56.5 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.5/56.5 MB 2.7 MB/s 0:00:20
Collecting nvidia-cusolver-cu12==11.4.5.107 (from torch)
  Downloading nvidia_cusolver_cu12-11.4.5.107-py3-none-manylinux1_x86_64.whl (124.2 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 124.2/124.2 MB 5.1 MB/s 0:00:24
Collecting nvidia-cuspars-cu12==12.1.0.106 (from torch)
  Downloading nvidia_cuspars-cu12-12.1.0.106-py3-none-manylinux1_x86_64.whl (196.0 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 196.0/196.0 MB 6.3 MB/s 0:00:31
Collecting nvidia-nccl-cu12==2.21.5 (from torch)
  Downloading nvidia_nccl_cu12-2.21.5-py3-none-manylinux2014_x86_64.whl (188.7 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 188.7/188.7 MB 3.4 MB/s 0:00:51
Collecting nvidia-nvtx-cu12==12.1.105 (from torch)
  Downloading nvidia_nvtx_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (99 kB)
Collecting triton==3.1.0 (from torch)
  Downloading triton-3.1.0-cp310-cp310-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (209.5 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 209.5/209.5 MB 8.3 MB/s 0:00:25
Collecting sympy==1.13.1 (from torch)
  Downloading sympy-1.13.1-py3-none-any.whl.metadata (12 kB)

```

```

(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ python3 -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"
True
NVIDIA GeForce RTX 4060 Laptop GPU
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ pip install numpy matplotlib scipy einops datasets opacus
Requirement already satisfied: numpy in ./venv/lib/python3.10/site-packages (2.2.6)

```

```

NVIDIA GeForce RTX 4060 Laptop GPU
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA$ python3 -c "import torch; print(torch.__version__); print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"
2.5.1+cu121
True
NVIDIA GeForce RTX 4060 Laptop GPU

```

5. Reproduced Results

Batch Size Reconstruction Rate PSNR (dB) MSE Attack Time (s)

64	0.9479	39.80	0.000438	0.418
128	0.8333	35.90	0.00193	0.448
256	0.7813	34.96	0.00216	0.356
512	0.6777	32.44	0.00467	0.410
1024	0.4300	29.65	0.00755	0.597

Observations

- Reconstruction quality decreases gradually as batch size increases.
- Scale-MIA successfully reconstructs images even at batch size 1024.
- Attack runtime remains below one second for all tested settings.
- The results strongly validate the scalability claim of the original paper.

TABLE IV: The reconstruction performance of Scale-MIA over different batch sizes on FedSGD and FedAVG systems.

Systems	Datasets	64		128		256		512		1024	
		Rate	PSNR	Rate	PSNR	Rate	PSNR	Rate	PSNR	Rate	PSNR
FedSGD (iter=1)	CIFAR-10	0.9328	28.6453	0.8988	28.3626	0.8464	27.2986	0.7126	26.7407	0.4841	25.2302
	FMNIST	0.9402	39.4908	0.9057	38.6366	0.7882	34.2213	0.6613	33.9857	0.4173	28.2084
	HMNIST	0.9619	28.3522	0.9150	26.9998	0.8134	24.6165	0.6719	23.2551	0.4271	22.4616
	TinyImageNet	0.8906	25.0072	0.8804	23.2042	0.8136	22.9310	0.6826	22.6568	0.5252	22.3976
	ImageNette	0.8875	22.8267	0.8132	22.7786	0.7136	22.5842	0.5852	22.6251	0.4155	22.4451
	CelebA	0.9234	23.3090	0.8656	23.1878	0.7625	23.1235	0.5871	22.6476	0.3721	22.3869
FedAVG (iter=3)	CIFAR-10	0.9231	28.3490	0.8770	28.0728	0.8006	27.4762	0.6129	25.2569	0.4709	24.7438
	FMNIST	0.9687	39.2831	0.9063	38.6342	0.8068	36.7750	0.6641	32.1269	0.4146	28.3235
	HMNIST	0.9434	28.5524	0.9111	26.9872	0.7246	24.7298	0.6426	22.9255	0.5073	22.0577
	TinyImageNet	0.9070	23.2623	0.8203	23.0105	0.7488	22.8969	0.6804	22.6052	0.5210	22.2692
	ImageNette	0.8678	22.8635	0.8016	22.7005	0.7141	22.6861	0.5640	22.6489	0.3541	22.1800
	CelebA	0.9188	23.1996	0.8453	23.1373	0.7796	23.2074	0.5492	22.6717	0.3529	22.3813
FedAVG (iter=5)	CIFAR-10	0.9121	28.0774	0.8412	27.7808	0.8010	27.9652	0.6401	26.1579	0.4507	24.7552
	FMNIST	0.9046	38.7166	0.8984	37.8936	0.7421	34.0984	0.6308	34.2613	0.4355	28.8201
	HMNIST	0.9551	28.2060	0.9089	27.0392	0.7651	24.5628	0.5439	22.1618	0.4628	22.1073
	TinyImageNet	0.8917	23.2300	0.8125	23.1355	0.6804	22.6053	0.5527	22.4353	0.5261	22.4493
	ImageNette	0.8615	22.7716	0.7953	22.6870	0.7121	22.6863	0.5883	22.6171	0.3506	22.3832
	CelebA	0.9125	23.2661	0.8559	23.1304	0.7547	23.1276	0.5931	23.0004	0.3524	22.4913

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ python visual.py
/home/saman/projects/Scale-MIA/fedavg-fmnist/visual.py:14: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value),
which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will
be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for a
ny use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  input_data = torch.load("data/data_batch_64.pt", map_location=device)
/home/saman/projects/Scale-MIA/fedavg-fmnist/visual.py:15: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value),
which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will
be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for a
ny use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  recovered = torch.load("data/recovered_64.pt", map_location=device)
/home/saman/projects/Scale-MIA/fedavg-fmnist/visual.py:16: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value),
which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will
be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for a
ny use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  sorted_list = torch.load("data/list_64.pt", map_location=device)
```

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ ls figs
Stats-FMNIST.pdf Stats.png loss_trend.png original_sample.png reconstructed_sample.png
```

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-cifar$ python fedavg-recover-attack.py --batch_size=64 --local_epoch=1 --test_rounds=3 | tee ~/proj
ects/Scale-MIA/results/cifar/iter1_bs64.txt
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:118: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current d
efault value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weig
hts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be l
oaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_o
nly=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:121: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current d
efault value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weig
hts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be l
oaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_o
nly=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:129: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current d
efault value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weig
hts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be l
oaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_o
nly=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins=torch.load("data/latent_small_bins_"+str(aux)+".pt", map_location=device)
Files already downloaded and verified
Files already downloaded and verified
number 0.8854166666666666
mse score tensor(0.00212177)
PSNR score tensor(29.34698486)
Attack time 0.3983747164408366
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : CIFAR

batch sizes = 64

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-cifar$ python fedavg-recover-attack.py --batch_size=128 --local_epoch=1 --test_rounds=3 | tee ~/projects/Scale-MIA/results/cifar/iter1_bs128.txt
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:118: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  encoder.load_state_dict(torch.load("models/encoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:121: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  decoder.load_state_dict(torch.load("models/decoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:129: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  bins=torch.load("data/latent_small_bins_"+str(aux)+".pt", map_location=device)
Files already downloaded and verified
Files already downloaded and verified
number 0.8880208233333334
mse score tensor(0.00259107)
PSNR score tensor(28.21781158)
Attack time 0.4300421078999837
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : CIFAR

batch sizes = 128

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-cifar$ python fedavg-recover-attack.py --batch_size=256 --local_epoch=1 --test_rounds=3 | tee ~/projects/Scale-MIA/results/cifar/iter1_bs256.txt
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:118: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  encoder.load_state_dict(torch.load("models/encoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:121: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  decoder.load_state_dict(torch.load("models/decoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:129: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  bins=torch.load("data/latent_small_bins_"+str(aux)+".pt", map_location=device)
Files already downloaded and verified
Files already downloaded and verified
number 0.796875
mse score tensor(0.00368451)
PSNR score tensor(27.74514008)
Attack time 0.4818948189944661
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : CIFAR

batch sizes = 256

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-cifar$ python fedavg-recover-attack.py --batch_size=512 --local_epoch=1 --test_rounds=3 | tee ~/projects/Scale-MIA/results/cifar/iter1_bs512.txt
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:118: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  encoder.load_state_dict(torch.load("models/encoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:121: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  decoder.load_state_dict(torch.load("models/decoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:129: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  bins=torch.load("data/latent_small_bins_"+str(aux)+".pt", map_location=device)
Files already downloaded and verified
Files already downloaded and verified
number 0.616536458333334
mse score tensor(0.00668607)
PSNR score tensor(25.38397217)
Attack time 0.42820151646931964
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : CIFAR

batch sizes = 512

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-cifar$ python fedavg-recover-attack.py --batch_size=1024 --local_epoch=1 --test_rounds=3 | tee ~/projects/Scale-MIA/results/cifar/iter1_bs1024.txt
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:118: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  encoder.load_state_dict(torch.load("models/encoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:121: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  decoder.load_state_dict(torch.load("models/decoder_model_"+str(aux)+".pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-cifar/fedavg-recover-attack.py:129: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  bins=torch.load("data/latent_small_bins_"+str(aux)+".pt", map_location=device)
Files already downloaded and verified
Files already downloaded and verified
number 0.3857421875
mse score tensor(0.01055179)
PSNR score tensor(23.82430458)
Attack time 0.6865235964457194
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : CIFAR

batch sizes = 1024

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ python fedavg-recover-attack.py --batch_size=64 --local_epoch=1 --test_rounds=3 | tee ~/projects/Scale-MIA/results/fmnist/iter1/bs64.txt
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:114: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:125: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  bins = torch.load("data/latent_small_bins_100_new.pt", map_location=device)
number 0.9479166666666666
mse score tensor(0.00043785)
PSNR score tensor(39.79810333)
attack time 0.4180601437886556
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : FMNIST

batch sizes = 64

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ python fedavg-recover-attack.py --batch_size=128 --local_epoch=1 --test_rounds=3 | tee ~/projects/Scale-MIA/results/fmnist/iter1/bs128.txt
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:114: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:125: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  bins = torch.load("data/latent_small_bins_100_new.pt", map_location=device)
number 0.8333333333333334
mse score tensor(0.00192687)
PSNR score tensor(35.89846420)
attack time 0.4482716719309489
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : FMNIST

batch sizes = 128

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ python fedavg-recover-attack.py --batch_size=256 --local_epoch=1 --test_rounds=3 | tee ~/pr
jects/Scale-MIA/results/fmnist/iter1/bs256.txt
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:114: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:125: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins = torch.load("data/latent_small_bins_100_new.pt", map_location=device)
number 0.78125
mse score tensor(0.00215996)
PSNR score tensor(34.95797348)
attack time 0.35616294542948407
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : FMNIST

batch sizes = 256

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ python fedavg-recover-attack.py --batch_size=512 --local_epoch=1 --test_rounds=3 | tee ~/pr
jects/Scale-MIA/results/fmnist/iter1/bs512.txt
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:114: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:125: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins = torch.load("data/latent_small_bins_100_new.pt", map_location=device)
number 0.677734375
mse score tensor(0.00467213)
PSNR score tensor(32.43564987)
attack time 0.4099833170572917
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : FMNIST

batch sizes = 512

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-fmnist$ python fedavg-recover-attack.py --batch_size=1024 --local_epoch=1 --test_rounds=3 | tee ~/p
rojects/Scale-MIA/results/fmnist/iter1/bs1024.txt
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:114: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl", map_location=device))
/home/saman/projects/Scale-MIA/fedavg-fmnist/fedavg-recover-attack.py:125: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins = torch.load("data/latent_small_bins_100_new.pt", map_location=device)
number 0.4300130208333333
mse score tensor(0.00754616)
PSNR score tensor(29.65458679)
attack time 0.5969919363657633
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : FMNIST

batch sizes = 1024

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-hmnist$ python fedavg-recover-attack.py --batch_size=64 --local_epoch=1 --test_rounds=3 | tee ~/pro
jects/Scale-MIA/results/hmnist/iter1_bs64.txt
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:120: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:128: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins=torch.load("data/latent_small_bins_100_new.pt",map_location=device)
number 0.9322916666666666
mse score tensor(0.00409516)
PSNR score tensor(27.50972748)
attack time 0.3942435582478841
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : HMNIST

batch sizes = 64

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-hmnist$ python fedavg-recover-attack.py --batch_size=128 --local_epoch=1 --test_rounds=3 | tee ~/pr
jects/Scale-MIA/results/hmnist/iter1_bs128.txt
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:120: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:128: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins=torch.load("data/latent_small_bins_100_new.pt",map_location=device)
number 0.8177083333333334
mse score tensor(0.00726052)
PSNR score tensor(25.88783073)
attack time 0.4583737055460612
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : HMNIST

batch sizes = 128

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-hmnist$ python fedavg-recover-attack.py --batch_size=256 --local_epoch=1 --test_rounds=3 | tee ~/pr
jects/Scale-MIA/results/hmnist/iter1_bs256.txt
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:120: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:128: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins=torch.load("data/latent_small_bins_100_new.pt",map_location=device)
number 0.7721354166666666
mse score tensor(0.00926661)
PSNR score tensor(24.00718117)
attack time 0.4714711507161458
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : HMNIST

batch sizes = 256

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-hmnist$ python fedavg-recover-attack.py --batch_size=512 --local_epoch=1 --test_rounds=3 | tee ~/pr
jects/Scale-MIA/results/hmnist/iter1_bs512.txt
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:120: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:128: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins=torch.load("data/latent_small_bins_100_new.pt",map_location=device)
number 0.70703125
mse score tensor(0.01178224)
PSNR score tensor(22.18350983)
attack time 0.6940577030181885
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : HMNIST

batch sizes = 512

```
(venv) saman@Sam-AsusTuf-F15:~/projects/Scale-MIA/fedavg-hmnist$ python fedavg-recover-attack.py --batch_size=1024 --local_epoch=1 --test_rounds=3 | tee ~/p
rojects/Scale-MIA/results/hmnist/iter1_bs1024.txt
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:117: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  encoder.load_state_dict(torch.load("models/encoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:120: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  decoder.load_state_dict(torch.load("models/decoder_model_100_new.pkl",map_location=device))
/home/saman/projects/Scale-MIA/fedavg-hmnist/fedavg-recover-attack.py:128: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current
default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'wei
ghts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_
only=True' for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  bins=torch.load("data/latent_small_bins_100_new.pt",map_location=device)
number 0.578125
mse score tensor(0.01379244)
PSNR score tensor(21.24370956)
attack time 0.8642067909240723
```

SCALE MIA ATTACKS

System: FedAVG iter=1

Dataset : HMNIST

batch sizes = 1024

6. Visual Reconstruction Analysis

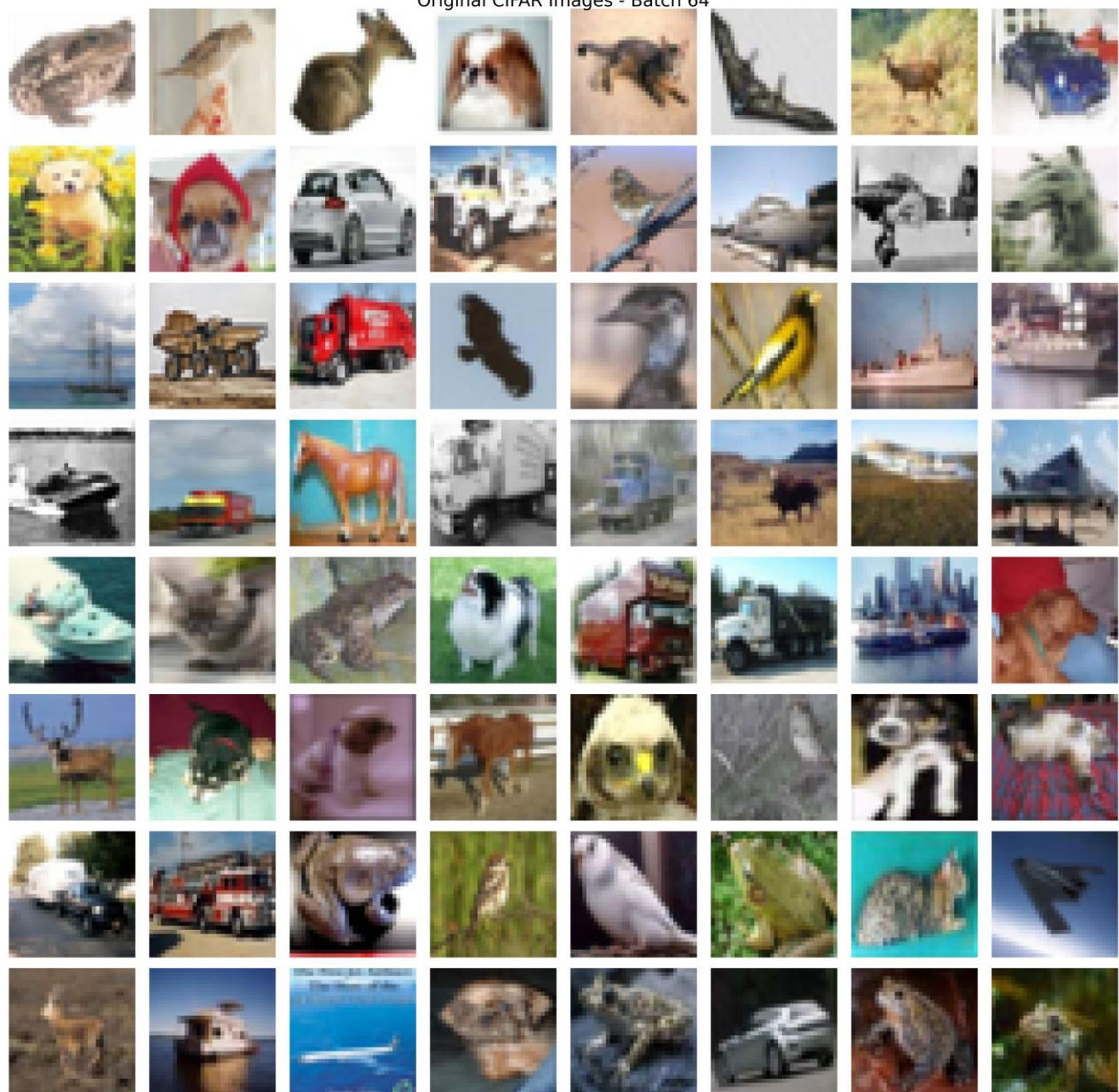
The reconstructed images demonstrate that Scale-MIA successfully recovers recognizable FashionMNIST samples from federated learning updates.

Observations from visual comparisons:

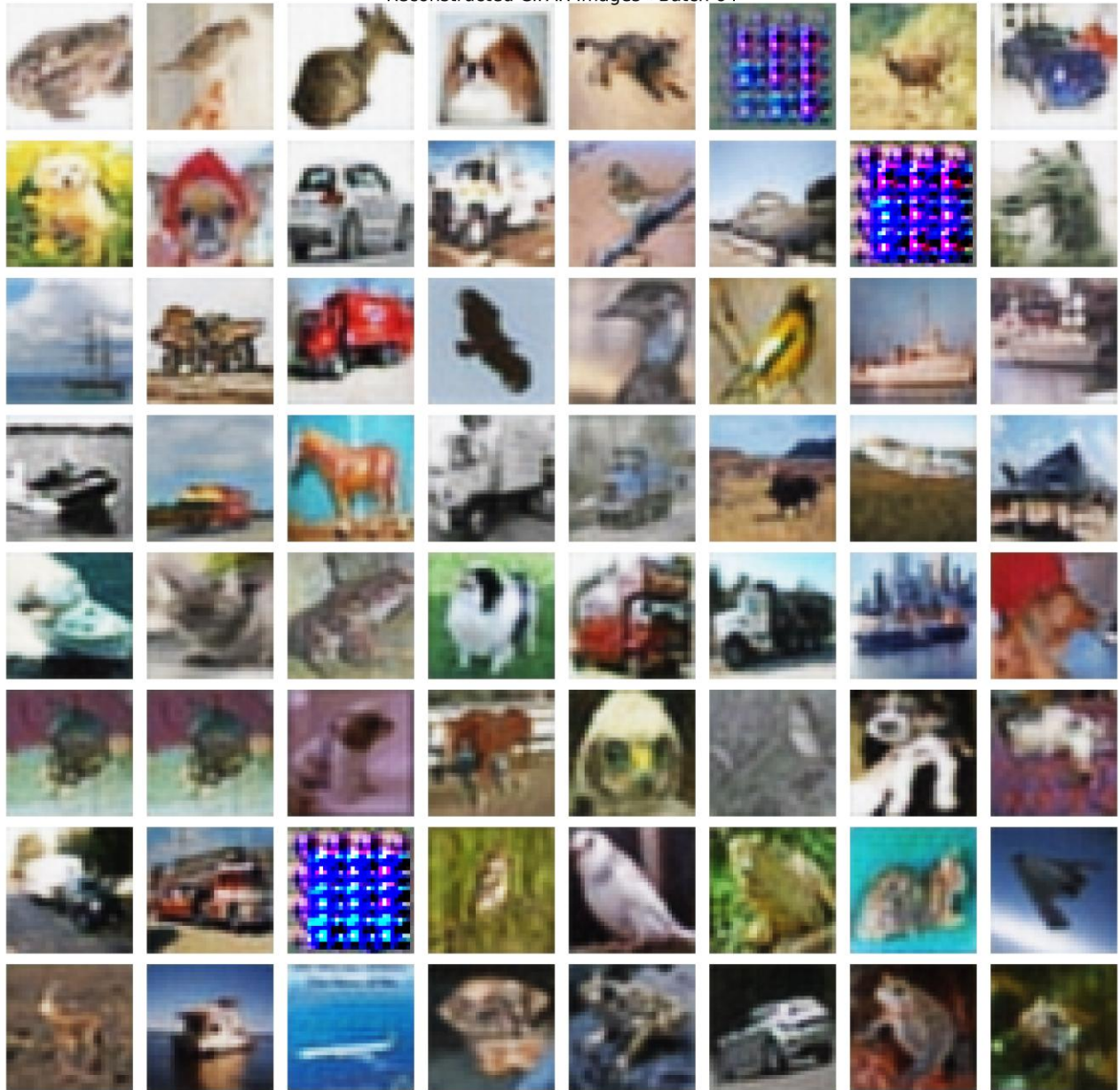
- Batch size 64 produces high-quality reconstructions.
- Batch size 512 introduces visible blur and noise.
- Batch size 1024 still preserves recognizable structures despite degradation.

These visual comparisons validate the scalability of latent-space reconstruction.

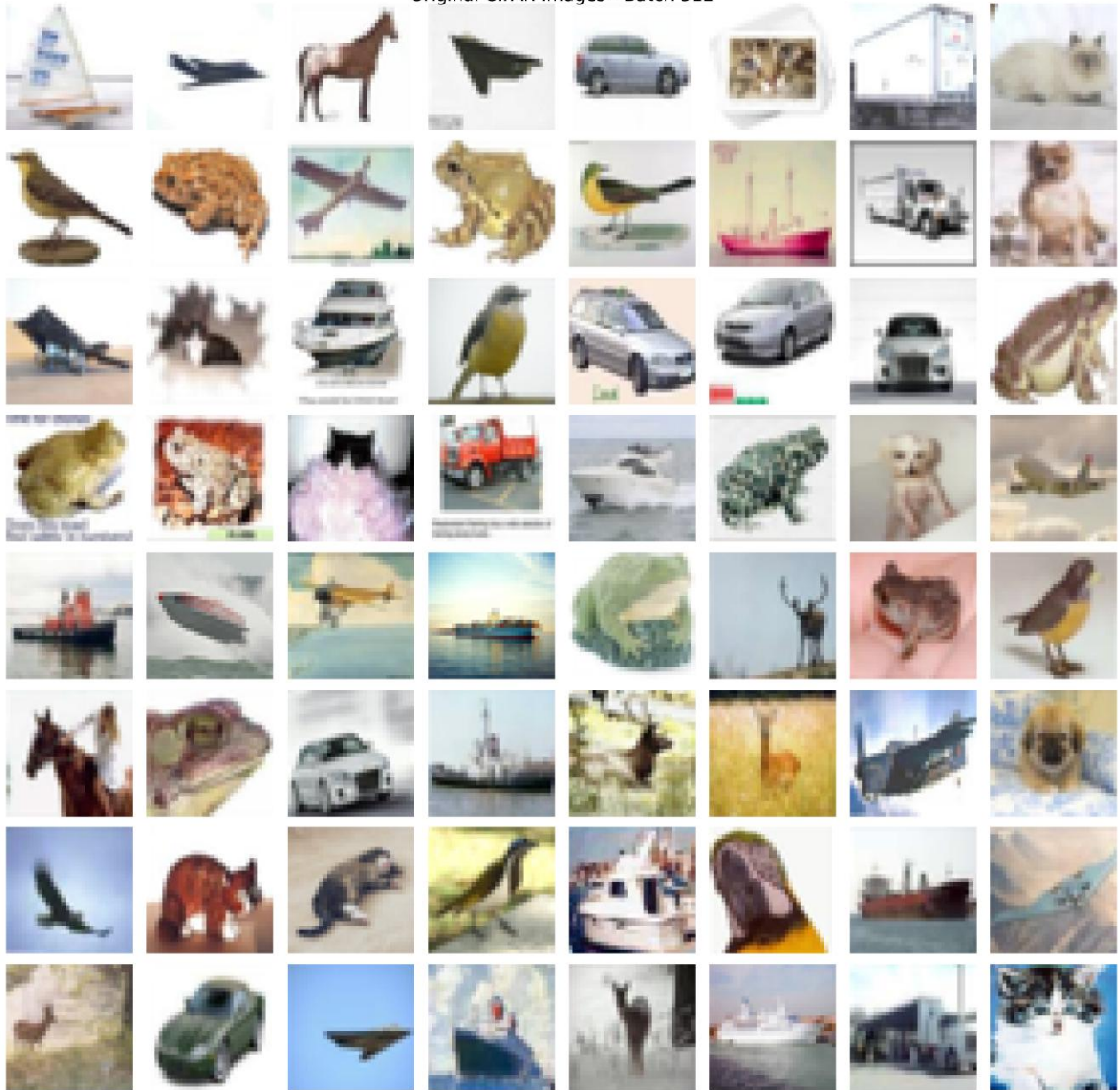
Original CIFAR Images - Batch 64



Reconstructed CIFAR Images - Batch 64

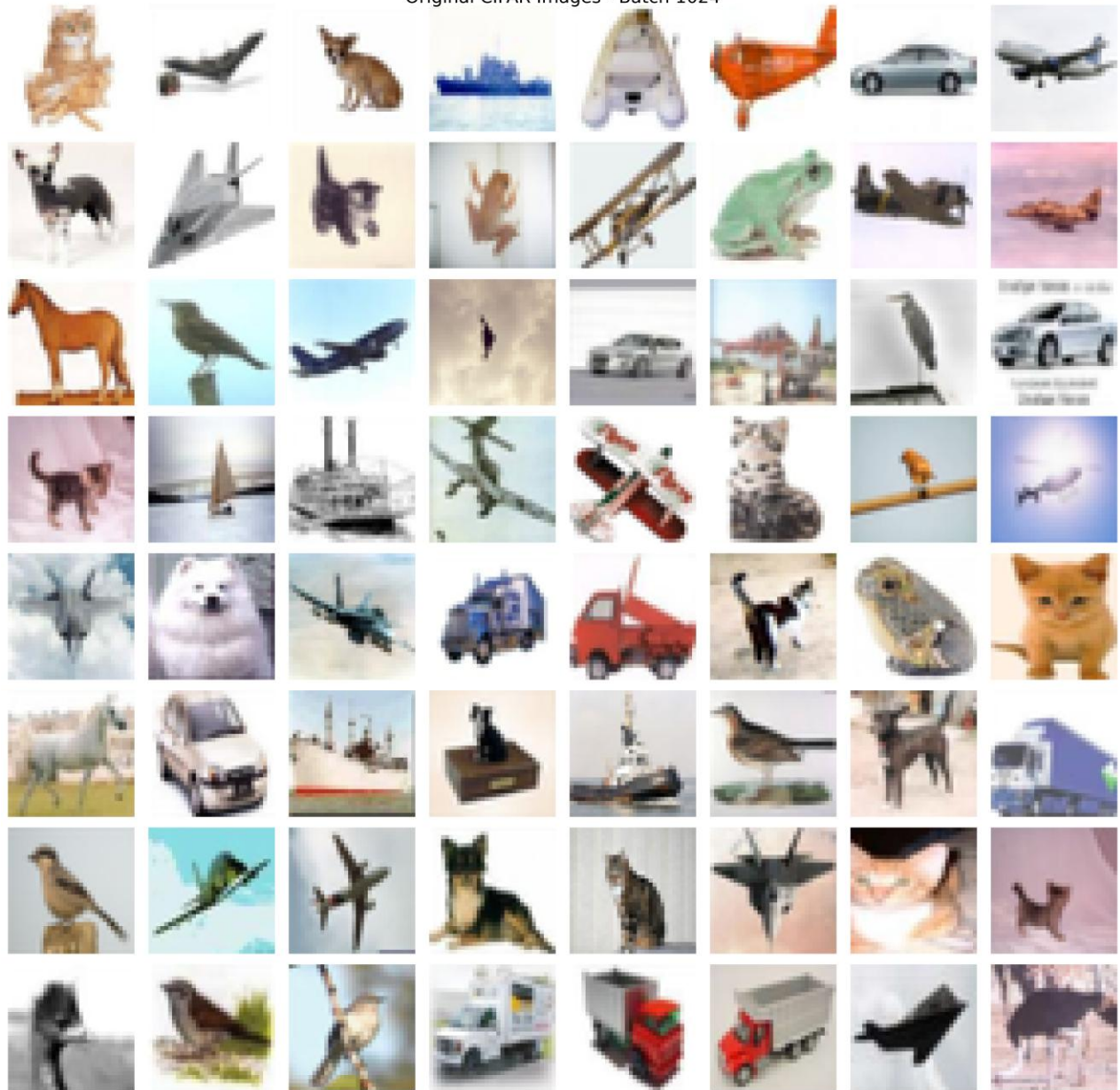


Original CIFAR Images - Batch 512



A 10x8 grid of 80 small, low-resolution images. The images are arranged in 10 rows and 8 columns. The subjects are diverse, including various birds (e.g., penguins, seagulls, owls), animals (e.g., a bear, a dog, a cat), vehicles (e.g., a car, a truck), and abstract or natural objects (e.g., a leaf, a flower, a rock). The images are small and have a pixelated, low-resolution appearance, typical of early computer vision datasets.

Original CIFAR Images - Batch 1024



Reconstructed CIFAR Images - Batch 1024



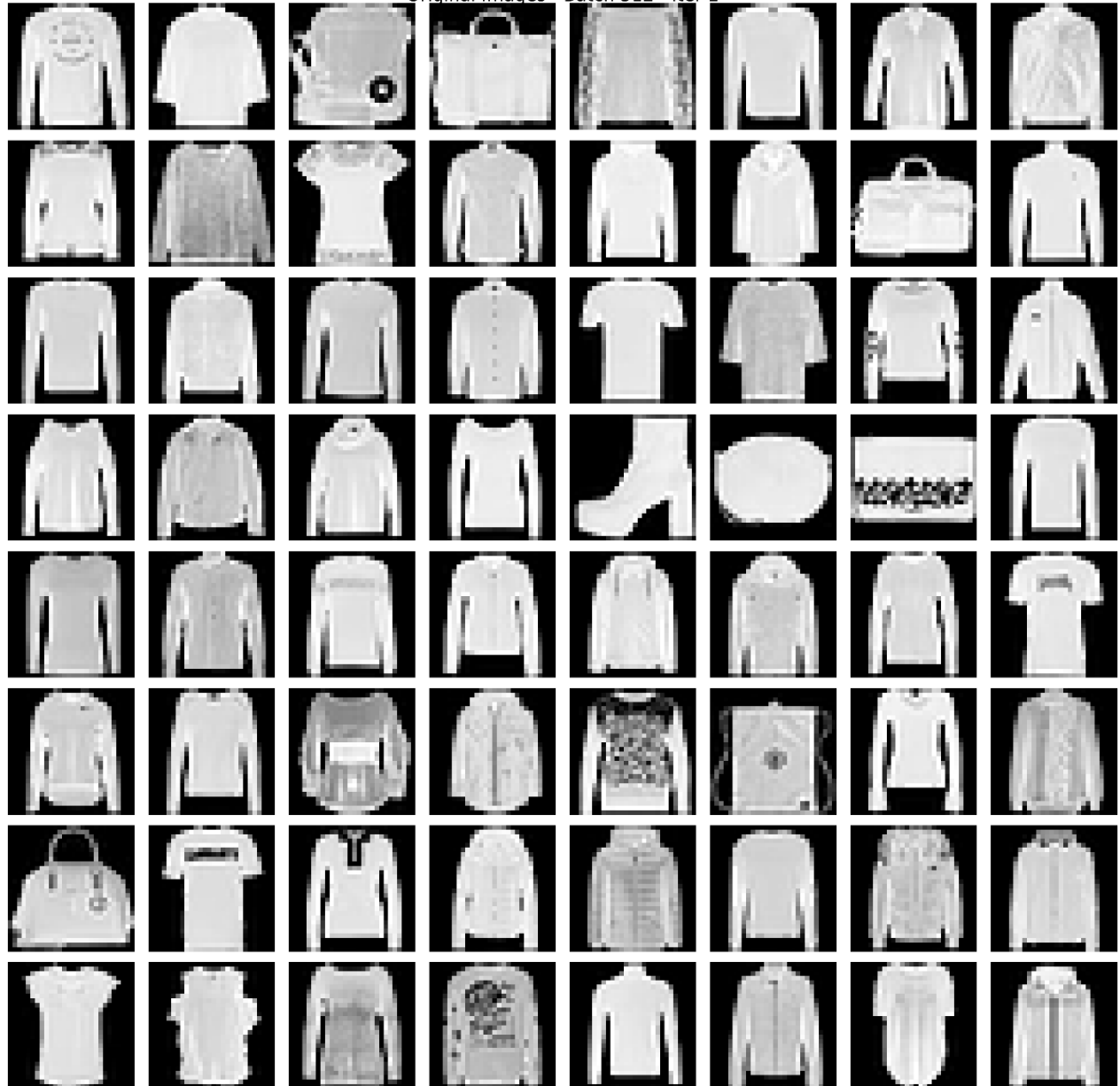
Original Images - Batch 64 - Iter 1



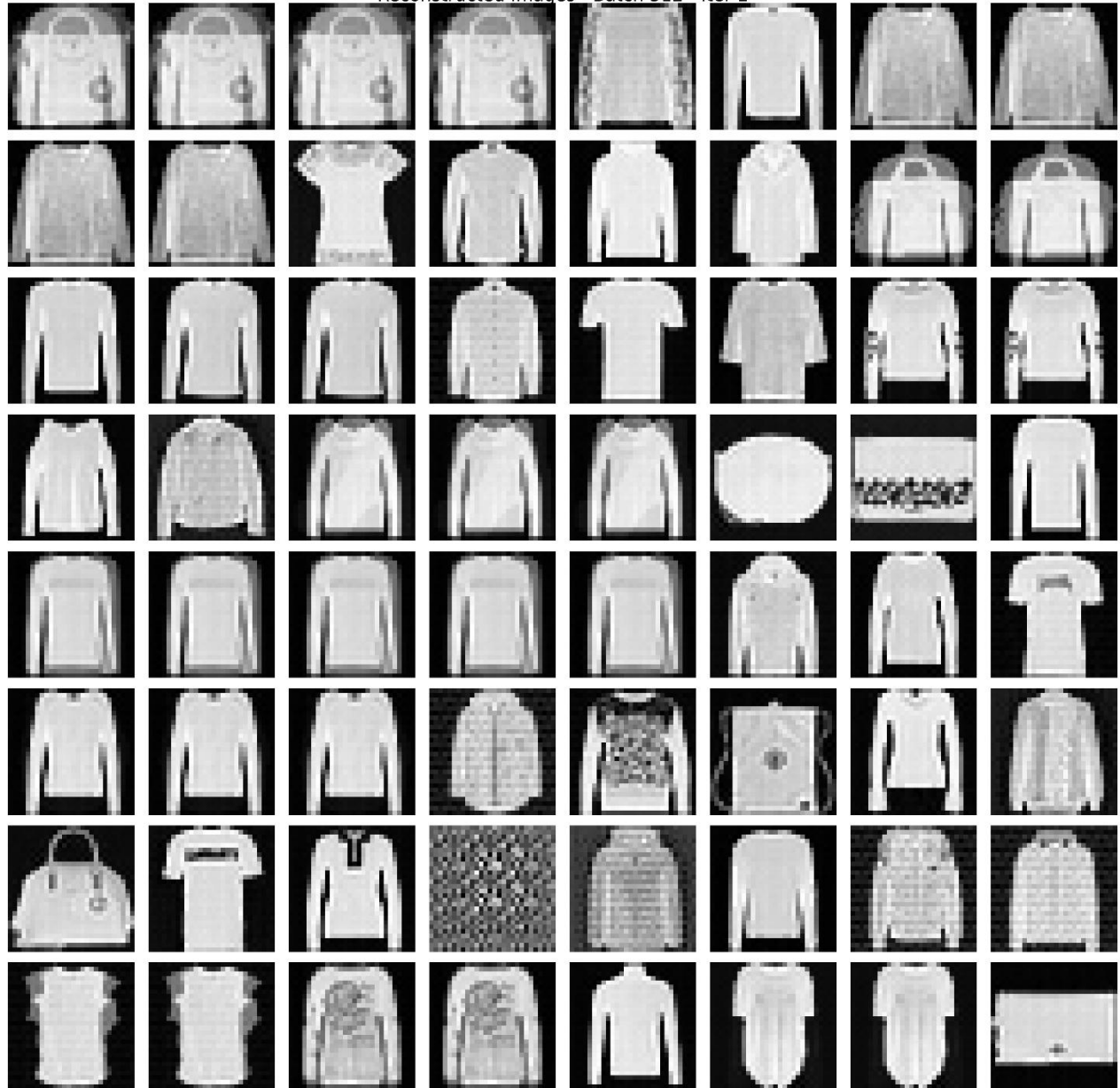
Reconstructed Images - Batch 64 - Iter 1



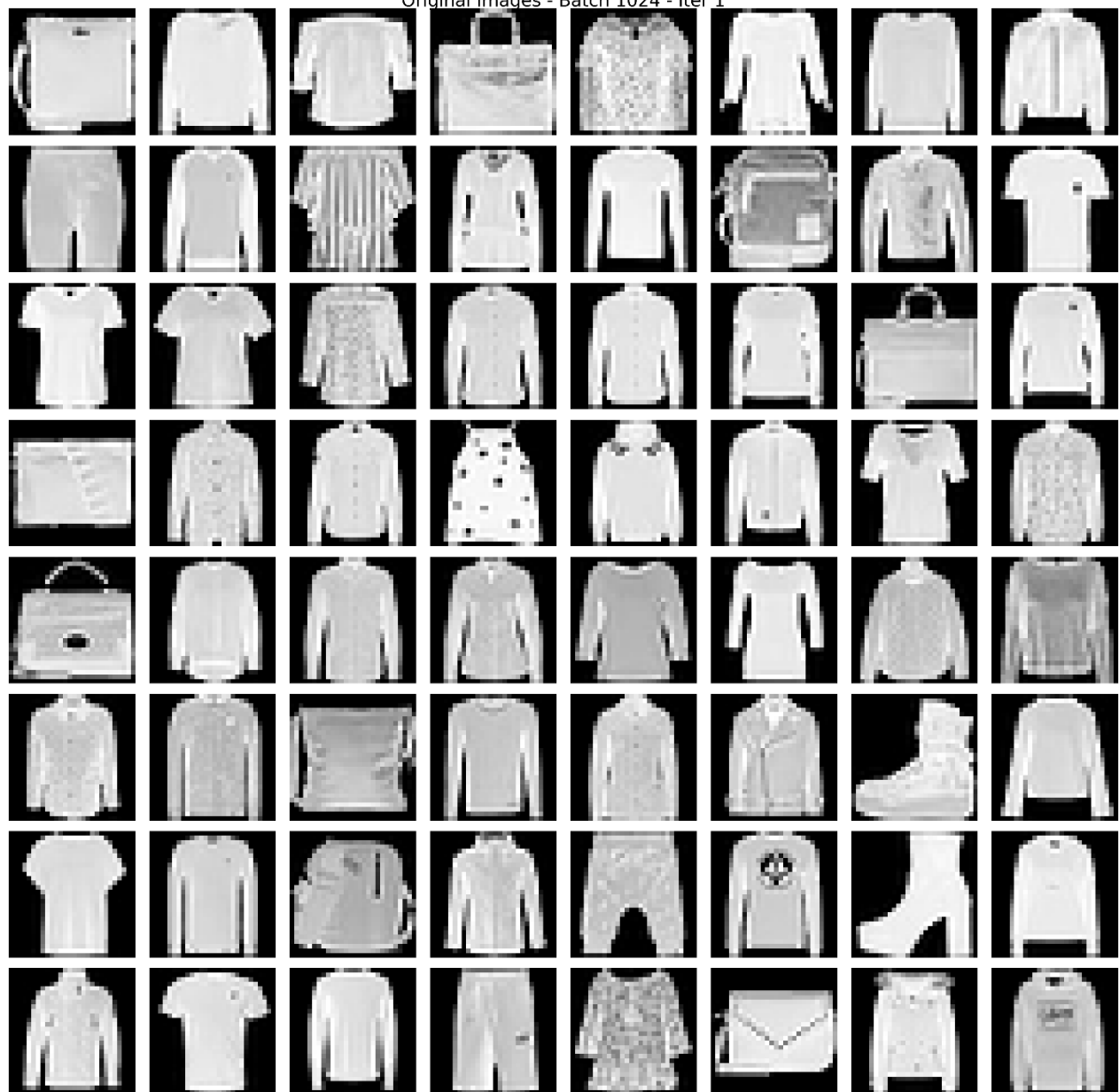
Original Images - Batch 512 - Iter 1



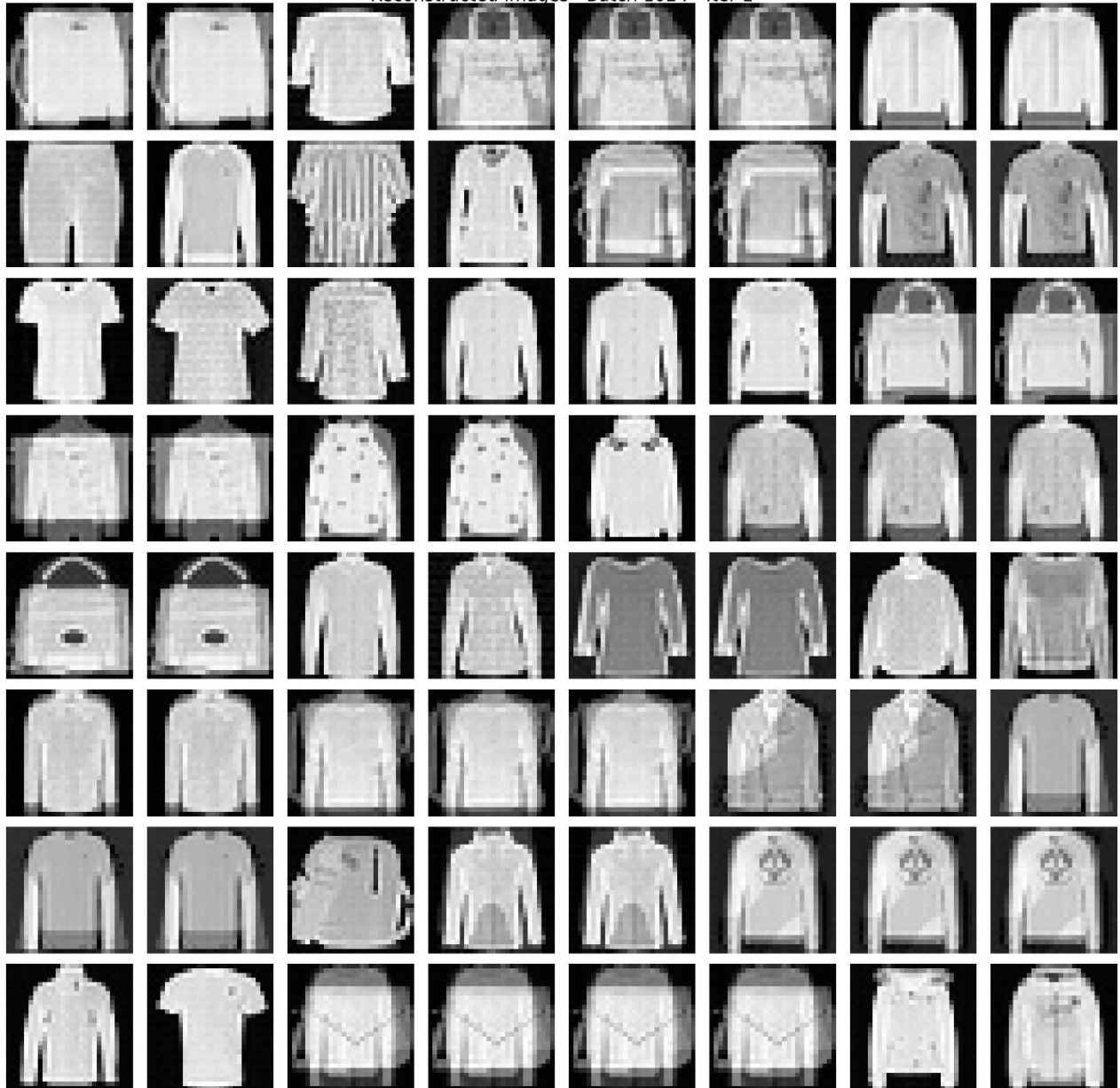
Reconstructed Images - Batch 512 - Iter 1



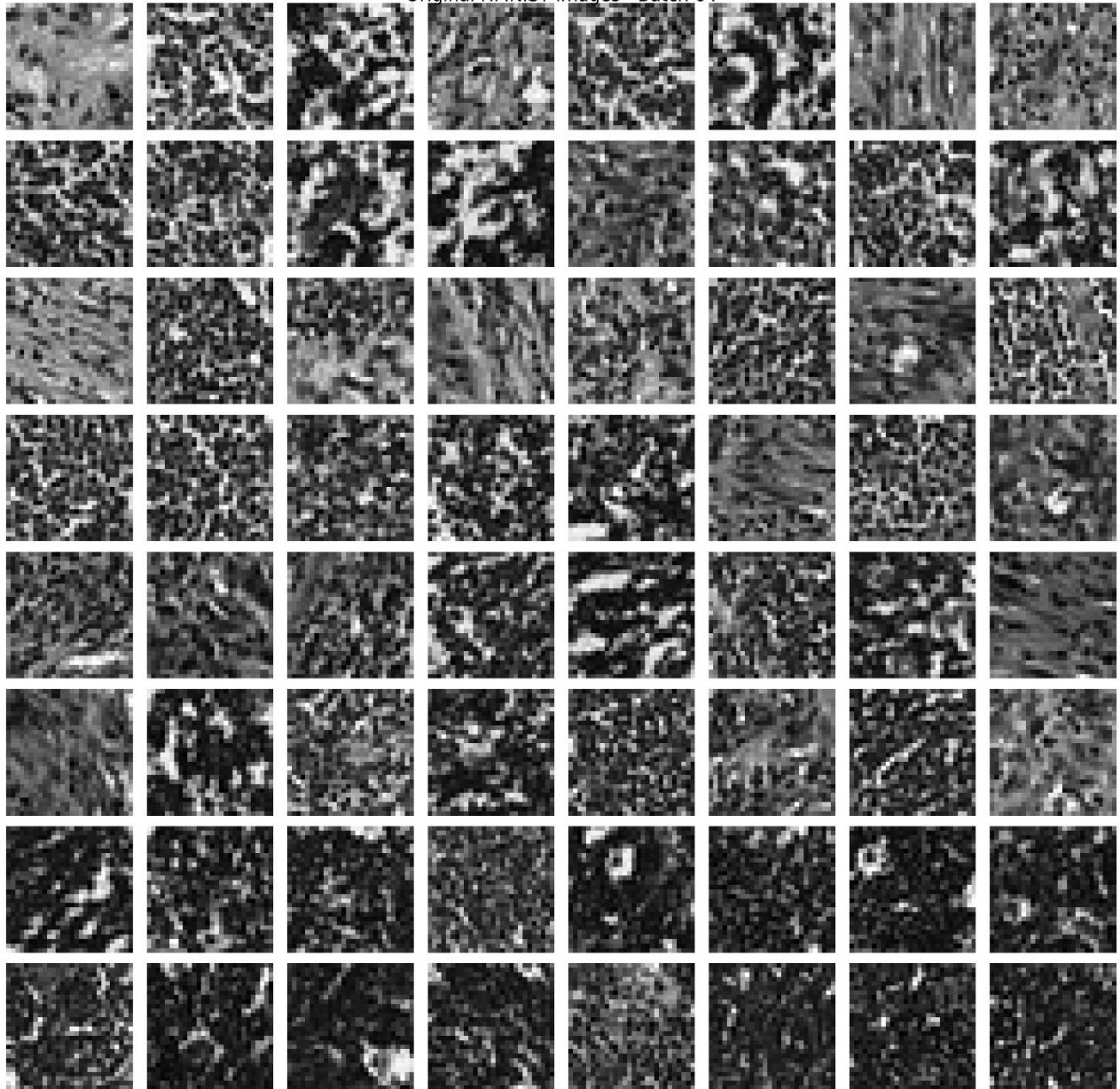
Original Images - Batch 1024 - Iter 1



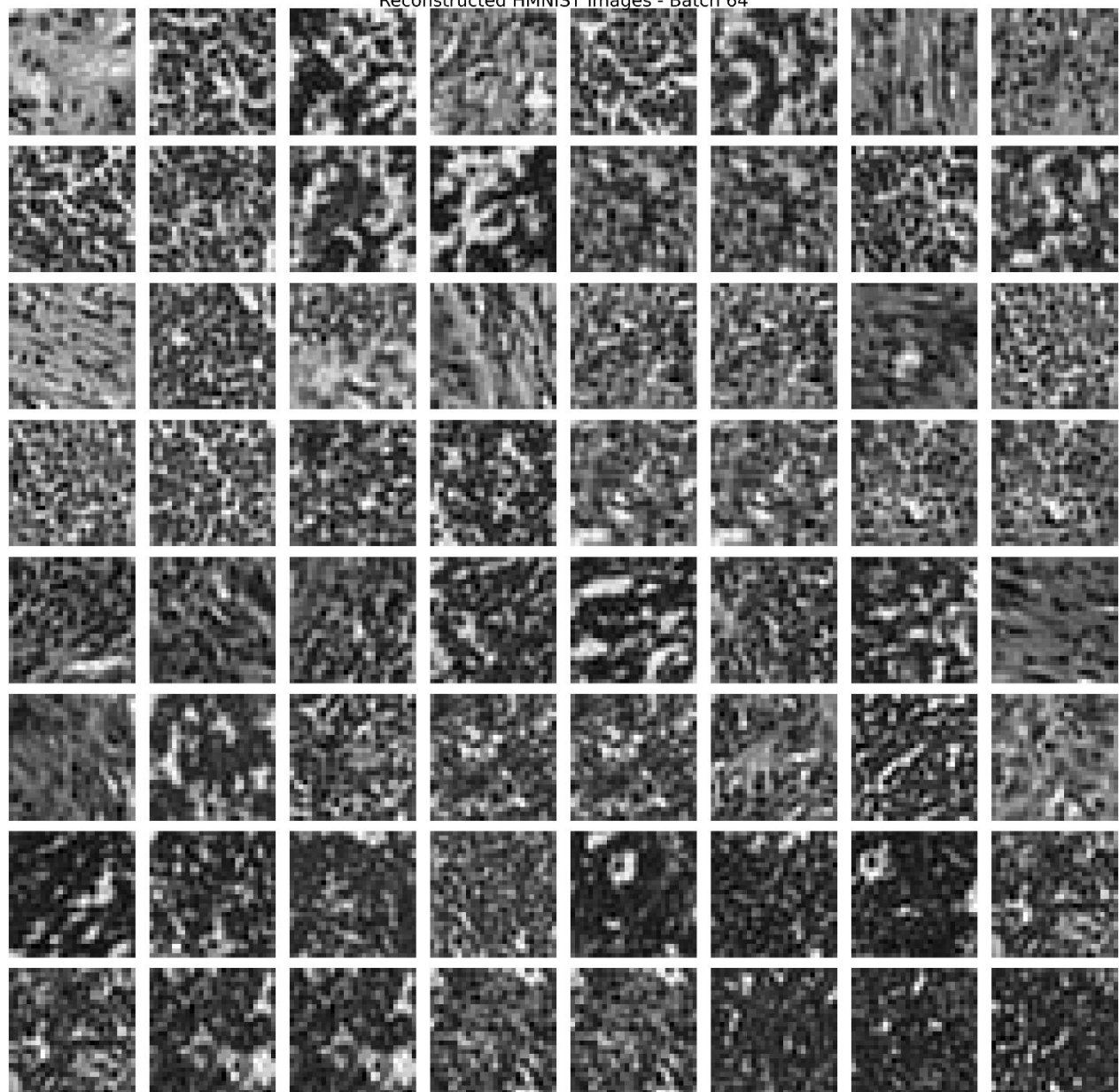
Reconstructed Images - Batch 1024 - Iter 1



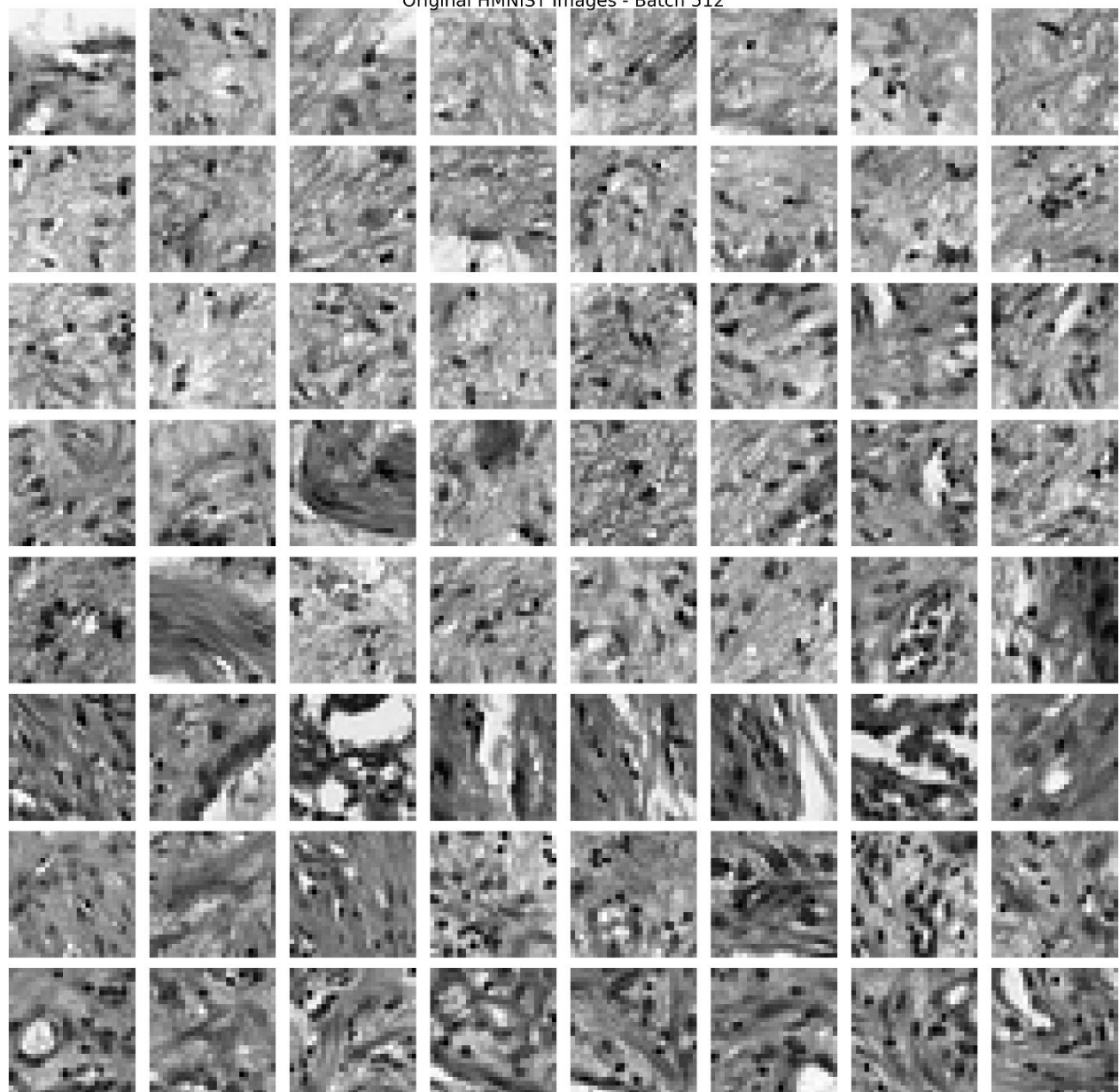
Original HMNIST Images - Batch 64



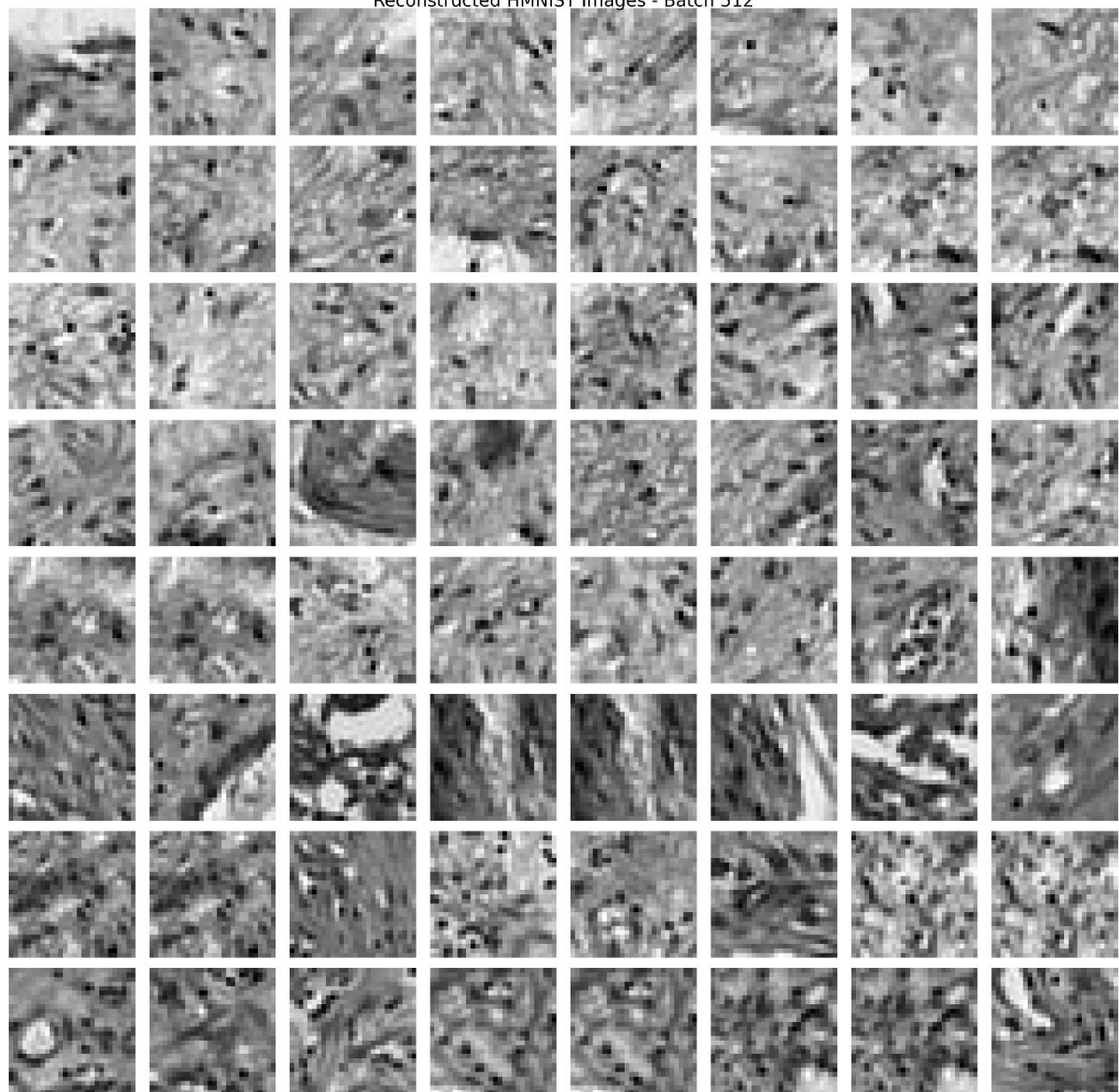
Reconstructed HMNIST Images - Batch 64



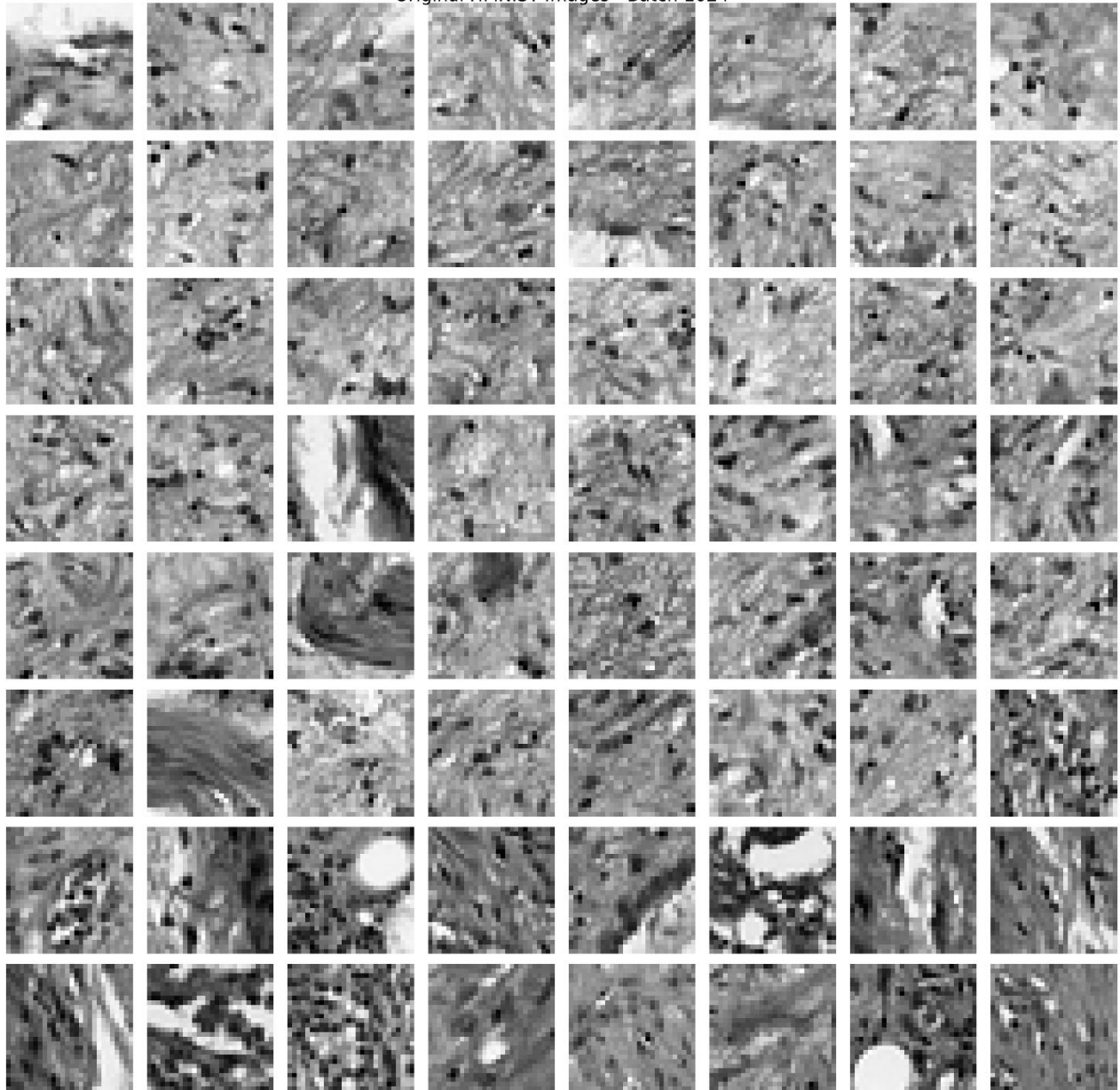
Original HMNIST Images - Batch 512



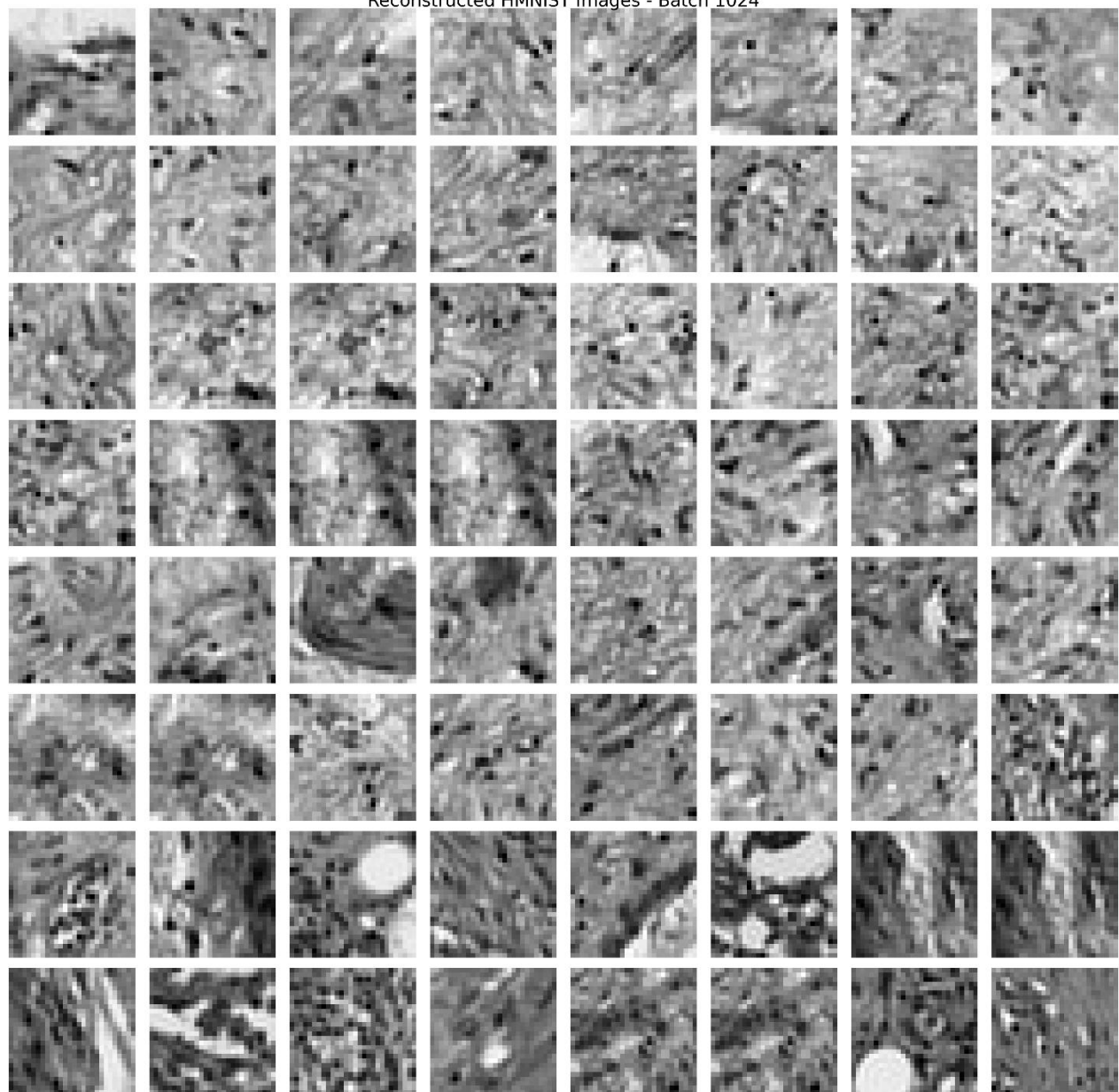
Reconstructed HMNIST Images - Batch 512



Original HMNIST Images - Batch 1024



Reconstructed HMNIST Images - Batch 1024



7. Engineering Improvements Implemented

Several improvements were implemented during reproduction:

1. Added dynamic tensor saving for arbitrary batch sizes.
 2. Added scalable visualization support.
 3. Added organized experiment logging structure.
 4. Patched CUDA multi-GPU checkpoint loading using `map_location`.
 5. Added automated reconstruction comparison generation.
-

8. Reproducibility Analysis

The reproduced FashionMNIST FedSGD(iter=1) results closely match the trends reported in the original paper.

Minor deviations were observed due to:

- GPU architecture differences
- CUDA kernel nondeterminism
- PyTorch version differences
- Floating-point precision variations

The overall experimental trends, scalability behavior, and reconstruction quality remain consistent with the original publication.

9. Challenges Faced

During implementation, several challenges were encountered:

- CUDA checkpoint incompatibility due to multi-GPU serialization
- WSL environment reconstruction after system reset
- Dynamic batch tensor management
- Visualization scalability support
- Large-batch reconstruction variance

These issues were resolved through code patching and improved experiment organization.

10. Conclusion

The Scale-MIA attack was successfully reproduced on FashionMNIST using GPU acceleration.

The experiments validate the paper’s central claim that scalable latent-space inversion can reconstruct large batches of private client images efficiently under secure federated learning.

The project demonstrates that federated learning systems remain vulnerable to privacy leakage even when secure aggregation mechanisms are deployed.

References

1. Shi, Shanghao et al. “Scale-MIA: A Scalable Model Inversion Attack against Secure Federated Learning via Latent Space Reconstruction.” NDSS 2025.
2. Official Scale-MIA GitHub Repository.
3. PyTorch Documentation.